

RAG (Retrieval-Augmented Generation)

Purpose: A detailed reference for learning RAG, preparing for AI/LLM engineering discussions, and applying RAG to QA automation and AI-agent workflows.

1. What is RAG?

RAG stands for Retrieval-Augmented Generation. It is an architecture in which an application retrieves relevant information from an external knowledge source and supplies that information to a Large Language Model (LLM) as context before the LLM generates an answer.

Simple definition:

RAG = Retrieve relevant information + Augment the prompt/context + Generate an answer

A typical RAG system separates knowledge preparation from question answering.

OFFLINE / INDEXING PHASE

```
Documents
  ↓
Loading / Extraction
  ↓
Cleaning
  ↓
Chunking
  ↓
Embedding
  ↓
Vector Database
```

ONLINE / QUERY PHASE

```
User Question
  ↓
Query Embedding
  ↓
Retrieval
  ↓
Relevant Chunks
  ↓
Augmentation
  ↓
LLM
  ↓
Generation
  ↓
Answer
```

2. Why do we need RAG?

A general LLM does not automatically know your private company documentation, current internal procedures, proprietary test cases, Jira history, or newly created engineering documents. RAG lets an application retrieve this information at query time.

- Use private or organization-specific knowledge without putting all of it into the model's parameters.
- Update knowledge by changing the source documents and index rather than retraining the LLM for every document change.
- Provide source context that can be inspected, filtered, cited, or evaluated.
- Search large knowledge bases without placing the entire knowledge base into the prompt.

3. The two major phases of RAG

3.1 Offline / ingestion phase

The system prepares knowledge before a user asks a question.

Raw data → Load → Clean → Chunk → Embed → Store in vector database

3.2 Online / query phase

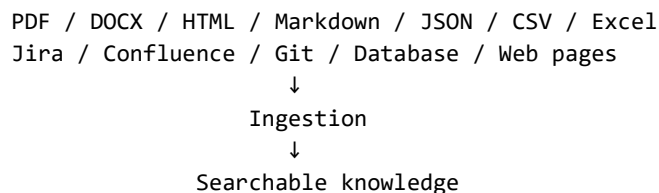
The system searches the prepared knowledge when the user asks a question.

Question → Embed query → Retrieve → Augment context → LLM → Generate answer

Key distinction: ingestion is generally a preparation/indexing activity; retrieval happens at query time.

4. What is ingestion?

Ingestion is the process of collecting data from source systems, extracting usable content, cleaning it, splitting it into useful units, attaching metadata, generating embeddings, and storing the resulting searchable representations.



Typical ingestion pipeline

- Connect to or load the source.
- Extract text or structured content.
- Clean unwanted content when appropriate.
- Chunk the content.
- Attach metadata such as source, page, product, version, date, or document type.
- Generate embeddings.
- Store vectors, text/chunk IDs, and metadata in the vector store.

5. What is chunking?

Chunking means dividing large documents into smaller units that can be independently retrieved. Good chunking tries to preserve enough meaning and context while keeping chunks small enough for accurate retrieval.

```
Document
-----
Chunk 1: [1 ----- 500]
Chunk 2:      [451 ----- 950]
Chunk 3:                [901 ----- 1400]
-----
Example: 500-token chunks with 50-token overlap
```

Common chunking strategies

- Fixed-size chunking: split by approximate token or character length.
- Sentence-based chunking: split at sentence boundaries.
- Paragraph/section chunking: preserve document structure.
- Recursive chunking: try larger structural boundaries first, then smaller ones.
- Semantic chunking: split when the topic or meaning changes significantly.
- Parent-child retrieval: retrieve a small child chunk but return a larger parent section for context.

6. What is metadata?

Metadata is information stored alongside a chunk that describes where it came from or how it should be filtered.

```
{
  "text": "Gateway message type 1 represents Gateway On...",
  "source": "gateway_documentation.pdf",
  "page": 42,
  "product": "OG Gateway",
  "version": "3.0"
}
```

Metadata can support filtering such as product = OG Gateway, version = 3.0, source = gateway_documentation.pdf, or date ranges.

7. What is an embedding?

An embedding is a numerical representation of data produced by an embedding model. It maps text, images, audio, or other supported content into a vector space where semantically related items tend to have related representations.

```
"Gateway firmware version"
↓
Embedding model
↓
[0.182, -0.421, 0.731, 0.092, ...]
```

The vector dimensions depend on the particular embedding model. Examples of embedding dimensions across different models can include hundreds or thousands of values.

Semantic similarity example

A = "How do I check gateway firmware?"
B = "Where can I find the gateway firmware version?"
C = "How do I cook pasta?"

A and B → likely close in embedding space
C → likely far away

8. What is a vector?

A vector is an ordered list of numerical values. An embedding is a vector generated by an embedding model to represent information in a machine-usable semantic space.

```
[0.12, 0.82, -0.44, 0.19]
```

9. What is a vector database?

A vector database or vector-capable data store stores embeddings and supports similarity search. A useful record often contains the vector plus the original chunk or a reference to it and metadata.

```
ID: 123
Text: "Gateway message type 1 contains heartbeat_enabled..."
Embedding: [0.123, -0.82, 0.44, ...]
Metadata: {"source": "gateway.pdf", "page": 42}
```

The system can search for vectors close to a query vector and return the corresponding chunks.

10. Why not use only a traditional database?

Traditional databases are excellent for exact structured queries. Semantic retrieval is useful when the wording of the question and the wording of the stored document are different but their meanings are related.

```
SQL-style:
SELECT * FROM employees WHERE department = 'QA';

Semantic search:
Question: "How do I verify firmware information?"
Possible document wording:
"The gateway reports its installed software version through firmware_version."
```

11. How vector retrieval works

```
User question
↓
Query embedding
↓
Query vector
↓
Similarity search
↓
Top-K relevant chunks
↓
Context for the LLM
```

Example conceptual scores:

```
Result A → similarity 0.91
Result B → similarity 0.84
Result C → similarity 0.79
Result D → similarity 0.32
Result E → similarity 0.21
```

12. Similarity metrics

- Cosine similarity: compares the angle/direction between vectors; widely used with embeddings.
- Euclidean distance: measures geometric distance; lower distance generally means closer vectors.
- Dot product: computes a vector product and is widely used by embedding/retrieval systems.

The appropriate metric and indexing configuration depend on the embedding model, vector store, and application.

13. What is retrieval?

Retrieval is the process of finding the most relevant information from the indexed knowledge base for the user's current question.

```
Question
↓
Query embedding
↓
Vector / hybrid search
↓
Candidate chunks
↓
Relevant context
```

14. What is Top-K?

Top-K means returning the K highest-ranked retrieval results. If $K = 3$, the retriever returns three candidates.

Top-K = 3

1. Gateway message type 1 schema
2. Gateway heartbeat configuration
3. Gateway message documentation

Too small a K can miss useful context. Too large a K can introduce irrelevant information, increase prompt size, and sometimes reduce answer quality.

15. What is augmentation?

Augmentation is the step where retrieved information is added to the LLM's input context, usually through a prompt template or structured context payload.

```
SYSTEM:
Answer using the supplied context.
```

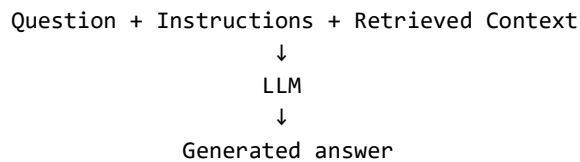
CONTEXT:
Gateway message type 1 contains:
- msg_type
- heartbeat_enabled
- heartbeat_period
- heartbeat_send_ack
- xbee_firmware_version

USER:
What does message type 1 contain?

The retrieved chunks are not the answer by themselves. They are evidence/context supplied to the generation model.

16. What is generation?

Generation is the LLM's production of the final response from the user's question, system instructions, and retrieved context.



17. Complete RAG example

Knowledge sources:
gateway_manual.pdf
gateway_api.md
gateway_test_cases.xlsx
gateway_logs.txt

Ingestion:
Load → extract → clean → chunk → embed → store

User question:
"Why is firmware_version None in the Gateway message?"

Retrieval:
Find schema + firmware mapping + backend processing chunks

Augmentation:
Put the relevant chunks into the LLM context

Generation:
LLM explains the issue using the retrieved evidence

18. RAG vs a normal LLM

Aspect	Normal LLM	RAG
External private knowledge	Not automatically available	Can retrieve from connected sources
Changing knowledge	Requires context or other	Index/source can be updated

	mechanism	
Search step	Not necessarily	Core part of architecture
Vector database	Not required	Common, but not mandatory
Answer context	Model/context window	Model + retrieved context

19. RAG vs fine-tuning

RAG	Fine-tuning	Typical use
Retrieves knowledge at query time	Changes model behavior/parameters through training	RAG: private/current documents
Knowledge can be updated by reindexing	Training process is needed for model changes	Fine-tuning: task/style behavior
Usually keeps source knowledge outside model weights	Knowledge/behavior is encoded in model parameters	Sometimes both are used together

20. Types of embeddings

There is no universally fixed number of embedding types. They can be classified by the modality or purpose being represented.

- Text embeddings: documents, paragraphs, queries, sentences.
- Sentence embeddings: compact representations of sentences.
- Document/section embeddings: representations of larger text units.
- Image embeddings: visual content and similarity.
- Audio embeddings: audio/speech-related representations.
- Video embeddings: video or temporal visual content.
- Multimodal embeddings: compatible representations across modalities such as text and images.

21. Embedding model providers and ecosystems

- OpenAI embedding APIs, including the text-embedding-3 family.
- Google embedding models available through Google's AI ecosystem.
- Cohere embedding models.
- Voyage AI embedding models.
- Hugging Face ecosystem containing many open and community models.
- Sentence Transformers ecosystem with many locally runnable embedding models.

Model names, pricing, dimensions, context limits, and availability change over time, so check the provider's current documentation before selecting a production model.

22. Free vs paid embeddings

There is no single free/paid split. A model may be open-source or openly licensed and runnable locally, while a hosted API may charge per usage.

Approach	Typical cost model	Trade-offs
Local open model	No per-request API fee; infrastructure still costs resources	More control, but you manage hardware/runtime
Hosted embedding API	Usually usage-based	Easy setup and scaling; data/API

		dependency
--	--	------------

23. What types of vector databases exist?

There is no universally fixed number of vector databases. The ecosystem can be grouped into several categories.

- Dedicated vector databases: designed primarily around vector search, such as Qdrant, Milvus, Weaviate, and Pinecone.
- Vector search libraries: such as FAISS. These provide similarity-search capabilities but are not equivalent to a complete database service.
- Traditional databases with vector extensions/features: PostgreSQL with pgvector, Redis, Elasticsearch/OpenSearch, and other systems.
- Local/developer vector stores: useful for prototypes and local RAG development.

24. Vector database examples: free/open vs managed

Technology	Category	Open source/open option	Local/self-hosted	Cloud/managed
FAISS	Vector search library	Yes	Yes	You manage hosting
Chroma	Vector store/database	Yes	Yes	Hosted options exist
Qdrant	Vector database	Yes	Yes	Yes
Weaviate	Vector database	Yes	Yes	Yes
Milvus	Vector database	Yes	Yes	Yes
Pinecone	Managed vector database	No typical self-hosted model	No typical self-hosted deployment	Yes
PostgreSQL + pgvector	Relational DB + vector extension	Yes	Yes	Depends on provider
OpenSearch	Search engine + vector capabilities	Yes	Yes	Yes

25. A practical beginner RAG stack

For learning, you can build a complete local RAG system without paying for every component.

```

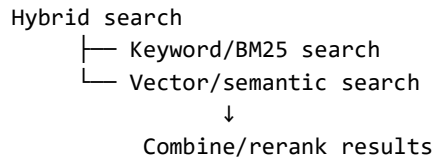
Python
+
Sentence Transformers / another local embedding model
+
Chroma or Qdrant
+
LangChain / LlamaIndex / Langflow
+
A local or hosted LLM

```

The exact combination is a design choice. Start simple and change components only when you have a concrete reason.

26. Hybrid search

Hybrid search combines semantic/vector retrieval with lexical or keyword retrieval. This is useful when exact identifiers, error strings, ticket IDs, version numbers, field names, or code symbols matter.



For engineering QA data, exact strings such as LA-578, firmware_version, msg_type: 1, or version numbers can be important retrieval signals.

27. Reranking

A first-stage retriever may return many candidate chunks. A reranker can score those candidates again and place the most relevant items at the top.

```
Vector/keyword retrieval → 20 candidates → reranker → top 5 context chunks
```

28. Query rewriting and multi-query retrieval

A system can rewrite an ambiguous user question into a more retrieval-friendly query, or generate several related queries and combine their results.

```
User:
"Why is firmware missing?"

Possible retrieval queries:
"gateway firmware_version field missing"
"Gateway message type 1 firmware"
"firmware_version None backend mapping"
```

29. Metadata filtering

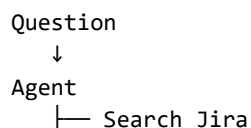
Metadata filtering restricts retrieval to records satisfying structured conditions.

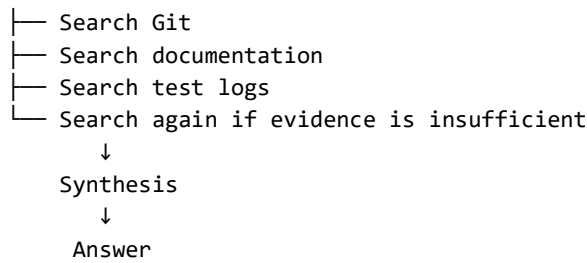
```
product = "OG Gateway"
version = "3.0"
document_type = "schema"
```

Filtering can reduce irrelevant retrieval and is particularly useful in multi-product or multi-version knowledge bases.

30. Agentic RAG

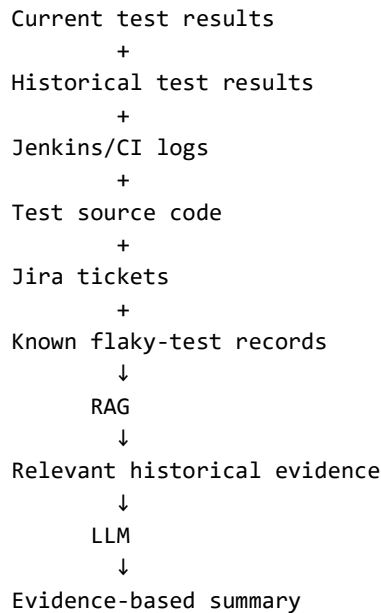
In agentic RAG, an AI agent can decide which tools or knowledge sources to query, whether another search is needed, and how to combine evidence. This is more dynamic than a fixed single-retriever pipeline.





31. RAG for a Flaky Test Finder

A useful QA-oriented RAG system could combine current test results with historical engineering knowledge.

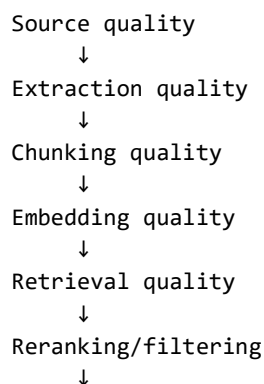


Example question: 'This test failed again. Have we seen a similar failure before?'

The retriever could find previous failures, similar log messages, related Jira tickets, previous fixes, and nearby test code. The LLM can then summarize those retrieved records. The important principle is that the model should distinguish retrieved evidence from speculation.

32. Important limitation: RAG does not automatically make answers truthful

RAG quality depends on the entire pipeline.



Prompt/context construction
 ↓
 LLM quality
 ↓
 Final answer

- Bad source data can produce bad answers.
- Bad chunk boundaries can hide necessary context.
- Poor retrieval can supply irrelevant evidence.
- Too much context can distract the model.
- The LLM can still make unsupported claims.
- Production RAG should be evaluated using retrieval and answer-quality metrics.

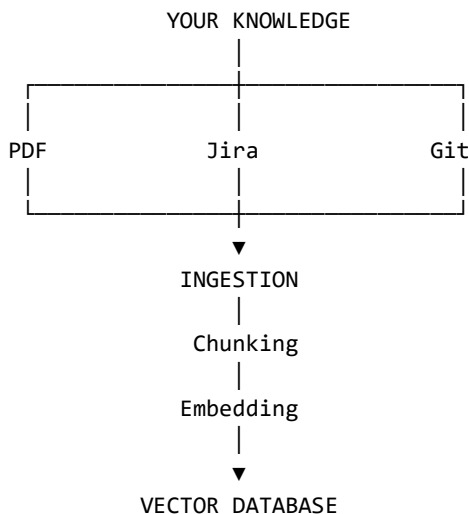
33. RAG evaluation: what to learn next

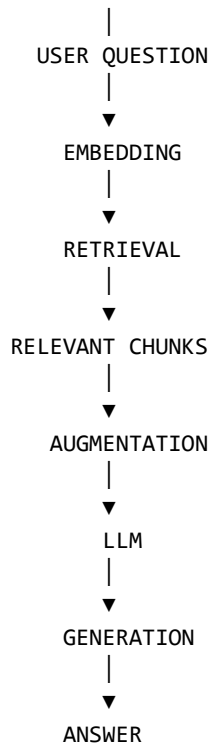
- Retrieval precision/recall and whether the needed chunk was retrieved.
- Context relevance: whether retrieved chunks actually relate to the question.
- Faithfulness/groundedness: whether the answer is supported by the supplied evidence.
- Answer relevance: whether the answer addresses the user's question.
- Latency and cost.
- Failure cases caused by chunking, filtering, embedding, retrieval, reranking, or generation.

34. The five terms to memorize

Term	Meaning
Ingestion	Prepare source data so it can be searched and used by the AI system.
Embedding	Convert content into numerical vectors representing useful semantic information.
Retrieval	Find relevant information for the current question.
Augmentation	Add retrieved information to the LLM's input context.
Generation	The LLM produces the final response from the question, instructions, and context.

35. One mental model





36. Interview-ready definition

RAG, or Retrieval-Augmented Generation, is an architecture that retrieves relevant external knowledge at query time and injects that knowledge into an LLM's context so the model can generate an answer grounded in the retrieved information. A typical RAG pipeline contains ingestion, chunking, embedding, vector storage, retrieval, context augmentation, and generation.

37. Short answers to the original questions

Question	Answer
What is RAG?	Retrieval-Augmented Generation: retrieve relevant external knowledge, add it to the LLM context, and generate an answer.
What is ingestion?	Preparing source data for retrieval: loading, extracting, cleaning, chunking, adding metadata, embedding, and indexing.
What is retrieval?	Finding the most relevant chunks or records for the user's query.
What is augmentation?	Adding retrieved evidence/context to the LLM input.
What is generation?	The LLM's production of the final response.
What is an embedding?	A numerical vector representation of content used to support semantic similarity/search.
What is a vector database?	A system that stores embeddings and supports efficient similarity/vector retrieval.
How many embedding types exist?	There is no fixed universal number. They can be grouped by modality such as text, image, audio, video, and multimodal embeddings.
How many vector databases exist?	There is no fixed universal number. There are dedicated vector databases, vector libraries, and traditional databases/search engines with vector

	capabilities.
Free or paid?	Both. Many open-source models and databases can be self-hosted; managed APIs/cloud services commonly charge based on usage or infrastructure.

38. Recommended learning sequence

1. LLM basics
2. Tokens and context windows
3. Embeddings
4. Vector similarity
5. Vector databases
6. Chunking strategies
7. Basic RAG pipeline
8. Retriever configuration / Top-K
9. Prompt augmentation
10. Hybrid search
11. Reranking
12. Metadata filtering
13. RAG evaluation
14. Agentic RAG
15. Production RAG: security, latency, cost, monitoring

39. Final cheat sheet

PREPARE → Ingestion
 CONVERT → Embedding
 STORE → Vector Database
 SEARCH → Retrieval
 ADD → Augmentation
 ANSWER → Generation

RAG = Search + Context + LLM

Embedding = Numbers representing content
 Vector DB = Store/search those vectors
 Retriever = Finds relevant information
 Augmentation = Gives that information to the LLM
 Generation = Produces the answer